ハッシュとシノニム

データ構造の基本である **ハッシュ法** と、データが重なったときの解決策である **「チェーン法」** および **「オープンアドレス法」** について、プログラミング初心者向けに最初から分かりやすく解説します。

---

1. ハッシュ法とは？（爆速でデータを探す仕組み）

通常、配列などのデータから目的のものを探すときは、先頭から1つずつ順番に調べていくため、データが増えるほど探す時間がかかります。

これに対し、ハッシュ法は「計算式を使ってデータの置き場所（部屋番号）を直接割り出し、一発でデータを取り出す技術」です。

身近なイメージ：学校の「出席番号」と「ロッカー」

学校で「Tanakaくんの荷物はどこ？」と探すとき、ロッカーを1つずつ開けて探すのは大変です。 もし「出席番号がそのままロッカーの番号」になっていれば、番号を見るだけで一発で目的のロッカーにたどり着けます。ハッシュ法はこの仕組みをプログラムで実現したものです。

---

2. データの置き場所を決める「ハッシュ関数」

置き場所を計算するルールのことを **ハッシュ関数** 、計算された部屋番号を **ハッシュ値** と呼びます。

例えば、**部屋が0番〜4番の5つ** あるシステムを考えます。

- **ルール（ハッシュ関数）：** 「数値を **5 で割ったあまり** を部屋番号とする」

このルールに従うと：

- `12` を入れる → $12 \div 5 = 2$ あまり `2` → **2番の部屋へ**
- `25` を入れる → $25 \div 5 = 5$ あまり `0` → **0番の部屋へ**

探すときも同じ計算をするだけで、「`12` は2番の部屋にある！」と一瞬で特定できます。

---

3. シノニムの発生（衝突 / コリジョン）とは？

ここで問題が発生します。

次に `17` というデータを入れようとしたとします。

- `17` を入れる → $17 \div 5 = 3$ あまり `2` → **2番の部屋へ**

しかし、2番の部屋にはすでに `12` が入っています。

このように、「違うデータなのに計算結果（ハッシュ値）が同じになり、置き場所がバッティングしてしまう現象」**を**シノニム発生（ハッシュ衝突 / コリジョン）と呼びます。

この衝突をどう解決するかで、主に2つの方法に分かれます。

---

4. 解決策①：チェーン法（Chaining）

チェーン法は、「部屋の中にデータを直接置くのではなく、数珠つなぎ（リスト）にしてぶら下げる」という方法です。

- **0番の部屋:** [25]
- **1番の部屋:** 空
- **2番の部屋:** [12] → [17] （数珠つなぎで連結！）
- **3番の部屋:** 空
- **4番の部屋:** 空

特徴

- **仕組み：** 衝突が起きたら、元々あったデータの「後ろ」に矢印（ポインタ）でどんどん繋いでいきます（まるで鎖＝チェーンのように繋がるためこう呼ばれます）。
- **メリット：** 部屋の数以上のデータが入ってきても、長さを伸ばしていけばいくらでも保存できます。
- **デメリット：** データの繋ぎ目を作るためにポインタ（メモリ領域）を追加で消費します。

---

5. 解決策②：オープンアドレス法（Open Addressing / 再ハッシュ法）

オープンアドレス法は、「部屋には1つのデータしか置かない。埋まっていたら別の空いている部屋を探す」という方法です。

空き部屋の探し方にはいくつかありますが、代表的な「すぐ隣の部屋を探す（線形探索法）」で見てみましょう。

1. 17 のハッシュ値を計算すると **2番**。
2. 2番の部屋を見に行くと 12 で埋まっている。
3. 「じゃあ隣の3番へ行こう」と移動する。
4. 3番は空いているので、そこに 17 を入れる。

- **0番の部屋:** [25]
- **1番の部屋:** 空
- **2番の部屋:** [12]
- **3番の部屋:** [17] （2番が埋まっていたので隣に入れた！）
- **4番の部屋:** 空

特徴

- **仕組み：** 衝突したら、あらかじめ決めたルール（隣の部屋を見る、一定数飛び越えて見るなど）で別の空き部屋を探して格納します。
- **メリット：** ポインタを使わないため、メモリを無駄なくシンプルに使えます。
- **デメリット：** データの削除処理が少し複雑になる点や、データが増えて空き部屋が減ると一気に性能が落ちる点があります。

---

6. まとめ

手法	衝突したときの対応	メリット	デメリット
チェーン法	同じ部屋に**リストで繋げて**ぶら下げる	データ数が増えても柔軟に対応できる	ポインタ用のメモリが必要
オープンアドレス法	別のアドレス（**空いている部屋**）を探して入れる	配列だけで管理でき、メモリ効率が良い	部屋が埋まってくると遅くなる